Adyuta AI - Developer Integration Guide

Welcome to the Adyuta AI integration documentation. This guide provides the necessary steps to connect your Flutter frontend application to the live Adyuta cloud backend.

1. Environment & API Access

Your application will communicate with a remote REST API.

- **Live Base URL:** `http://16.170.143.195`
- **API Documentation (Swagger UI):** `http://16.170.143.195/docs`

> **Note:** The API Documentation link above is an interactive Swagger portal. You can view all available endpoints, required request bodies, response schemas, and test API calls directly in your browser.

2. Incorporating the Frontend Source Code

You have been provided with the frontend source code (the Flutter UI and API service layers). To integrate this into your existing workspace:

1. **Copy the Source Files:** Move the contents of the provided `lib/` directory into your Flutter project's `lib/` folder.
2. **Copy Assets:** If an `assets/` folder was provided, merge it into your project root and declare the assets in your `pubspec.yaml`.
3. **Merge Dependencies:** Open the provided `pubspec.yaml` and copy the required packages (e.g., `http`, `shared_preferences`) into your project's `pubspec.yaml`. Run `flutter pub get`.

3. Configuration & Base URL

The provided Flutter code relies on a central configuration file to route HTTP requests.

Navigate to `lib/config.dart` (or the respective configuration file in the provided source) and verify that the `kDefaultBaseUrl` is set to the live server:

```dart
const String kDefaultBaseUrl = String.fromEnvironment(
  'ADYUTA_BASE_URL',
  defaultValue: 'http://16.170.143.195',
);
```

4. Platform-Specific Network Settings

Because the development API is currently utilizing HTTP (cleartext) rather than HTTPS, you must configure your mobile operating systems to allow cleartext network traffic.

Android Setup

Open `android/app/src/main/AndroidManifest.xml`. Ensure you have the internet permission, and add the `usesCleartextTraffic` property to the `<application>` tag:

```xml
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        ...
        android:usesCleartextTraffic="true">
    </application>
</manifest>
```

iOS Setup (If applicable)

Open `ios/Runner/Info.plist` and add the App Transport Security exception to allow HTTP loads:

```xml
<key>NSAppTransportSecurity</key>
<dict>
    <key>NSAllowsArbitraryLoads</key>
    <true/>
</dict>
```

5. Core Endpoints Overview

While the interactive `/docs` portal provides the full schema, here are the primary endpoints your UI will interact with:

Endpoint	Method	Description
`/capabilities`	`GET`	Returns a JSON manifest of all active AI/ML components and their current availability status.
`/router/health`	`GET`	Health check to verify connectivity to the routing layer.
`/rag/answer`	`POST`	The primary Conversational RAG endpoint. Accepts a user `question` and returns a generated, context-grounded answer.

6. Testing the Integration

Once the code is merged and permissions are set:

1. Launch the app on your emulator or physical device.
2. Navigate to the chat interface.
3. Send a test message. If the UI populates with a generated response, the frontend-to-backend pipeline is successfully integrated.